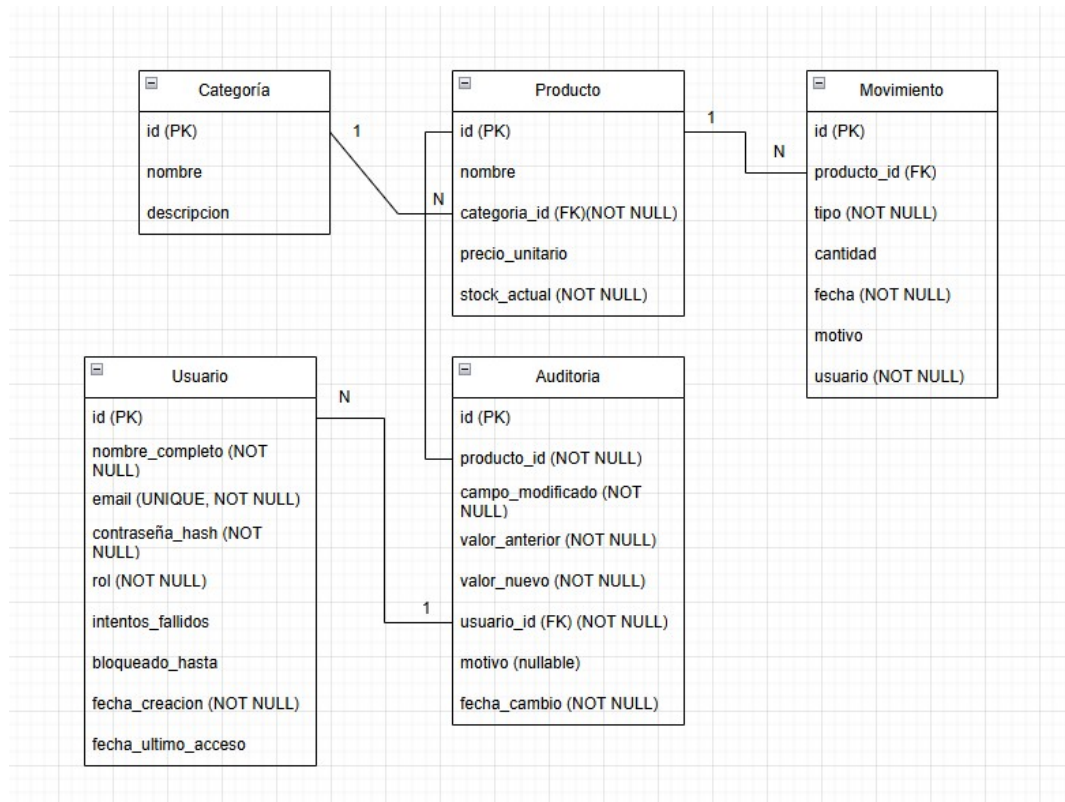


1.1 Diseño del modelo de datos

Proponé cómo estructurarías estas tres tablas. Podés hacerlo con un DER, un script SQL, modelos de ORM o como prefieras.



¿Qué tipo de base de datos elegirías y por qué?

Para este sistema de gestión de inventario, elegiría **PostgreSQL**. Mis razones principales son:

- **Relaciones Estrictas e Integridad:** Un inventario depende de vínculos claros y consistentes (un producto pertenece a una categoría, un movimiento se asocia a un usuario, etc.). Como motor relacional, PostgreSQL garantiza que estas reglas se cumplan siempre.
- **Escalabilidad y Robustez:** Es un motor extremadamente potente. Puede manejar de manera eficiente desde una base de datos pequeña hasta volúmenes de terabytes, lo cual es vital para el escenario de crecimiento a 500,000 productos planteado.
- **Extensibilidad:** Soporta tipos de datos avanzados y herramientas de indexación que permiten optimizar consultas complejas, algo indispensable para reportes históricos de stock.

¿Usarías un ORM? ¿Qué ventajas y desventajas ves en este caso?

Sí, optaría por utilizar un **ORM** (como **SQLModel** en conjunto con **SQLAlchemy**). Esta decisión se fundamenta en tres pilares:

- **Desarrollo Ágil y Seguro:** En lugar de escribir consultas SQL nativas manualmente, utilizo métodos del lenguaje de programación que ya conozco. Esto permite contar con **autocompletado** y **tipado fuerte**, reduciendo drásticamente los errores de sintaxis y previniendo ataques de **SQL Injection** de forma nativa.
- **Gestión de Migraciones (Control de Versiones de la DB):** El ORM facilita el uso de herramientas como **Alembic**. Esto actúa como un "Git" para la base de datos: permite llevar un historial exacto de los cambios en las tablas y sincronizar la estructura entre los distintos entornos (desarrollo, testing y producción) de forma automática.
- **Validación de Datos:** El ORM actúa como un filtro previo; asegura que los tipos de datos sean correctos antes de que la consulta llegue siquiera al motor de la base de datos, mejorando la calidad de la información almacenada.
- **Independencia del Motor:** Si en el futuro fuera necesario migrar a otro motor de base de datos, el ORM se encarga de realizar la "traducción" del esquema, minimizando el impacto en el código fuente.

Como desventaja menor, reconozco que un ORM puede añadir una leve capa de sobrecarga en consultas extremadamente complejas, pero para este sistema, los beneficios en mantenibilidad y velocidad de desarrollo superan ampliamente este costo.

¿Cómo garantizarías que el stock no quede negativo?

Para asegurar la integridad absoluta del inventario, aplicaría una estrategia de **defensa en capas**:

1. **Restricción en la Base de Datos (Última línea de defensa):** Implementaría un **CHECK Constraint** directamente en la tabla de productos. Esto garantiza que, a nivel físico, la base de datos rechace cualquier operación que resulte en un valor menor a cero.
 - *Ejemplo técnico:* ALTER TABLE producto ADD CONSTRAINT stock_no_negativo CHECK (stock >= 0);
2. **Lógica en el Backend (Capa de Servicio):** Antes de persistir cualquier movimiento, el servicio validará que el stock disponible sea suficiente. Además, utilizaría **Transacciones SQL** para manejar la concurrencia, evitando que dos ventas simultáneas sobre un mismo producto generen inconsistencias.
3. **Validación en el Frontend (Experiencia de Usuario):** En la interfaz, deshabilitaría el botón de "Confirmar Salida" y mostraría un mensaje de advertencia si la cantidad ingresada supera el stock actual disponible, evitando que el usuario envíe una petición fallida al servidor.

1.2 Convivencia y Orquestación del Stack Tecnológico

¿Cómo conviven estas herramientas dentro del stack?

En este proyecto, las herramientas no funcionan de forma aislada, sino que forman una cadena de responsabilidades bien definida para asegurar la integridad y la evolución del sistema:

- ORM (SQLModel / SQLAlchemy): Actúa como el motor de comunicación diario. Su función principal es traducir los objetos de nuestro código (Backend) a sentencias SQL de forma segura, evitando errores de sintaxis y protegiendo al sistema de inyecciones maliciosas.
- Migraciones (Alembic): Funcionan como el "control de versiones" de la base de datos. Utilizaría Alembic para registrar cada cambio estructural en las tablas (Categoría, Producto, Movimiento). Esto permite que la base de datos evolucione de forma segura, asegurando que todos los entornos (local, desarrollo y producción) sean idénticos sin riesgo de pérdida de datos.
- Motor de Base de Datos (PostgreSQL): Es el receptor final. No solo almacena la información, sino que aplica las restricciones físicas (Check Constraints) que garantizan que el stock nunca sea negativo, actuando como el último muro de defensa.

¿Cómo interactúan con el resto de la arquitectura?

Para que el sistema sea escalable y fácil de mantener, las herramientas interactúan mediante una Arquitectura por Capas. Esto permite que el cambio en una herramienta no rompa el resto del sistema:

1. Capa de Controlador (API): Es el punto de entrada. Recibe las peticiones HTTP (JSON) desde el frontend y valida que el formato sea correcto. Su única responsabilidad es delegar la tarea a la capa siguiente.
2. Capa de Servicio (Lógica de Negocio): Es el "cerebro" del sistema. Aquí se orquestan las Transacciones. Por ejemplo, cuando se registra una salida, el Servicio asegura que la creación del registro en la tabla Movimiento y la actualización del stock_actual en la tabla Producto ocurran como una operación atómica (o se hacen las dos, o no se hace nada). También es donde se valida la lógica de negocio, como verificar que haya stock suficiente antes de proceder.
3. Capa de Repositorio (Acceso a Datos): Es la única capa que tiene contacto directo con el ORM. Al encapsular el acceso a datos aquí, logramos un sistema desacoplado: si el día de mañana decidimos optimizar una consulta SQL o cambiar el motor de base de datos, solo modificamos esta capa sin afectar la lógica de negocio ni los controladores.

¿Cliente directo en algún caso?

Si bien el ORM cubre casi la totalidad de las necesidades, consideraría el uso de SQL Nativo (Cliente directo) únicamente en casos de reportes analíticos extremadamente complejos sobre los 500,000 productos, donde se requiera un nivel de optimización que el ORM no pueda alcanzar por sí solo. Sin embargo, para la gestión diaria del inventario, el ORM es la opción más segura y eficiente.

1.2 Historial de cambios

Si alguien edita el nombre o precio de un producto, ¿cómo guardarías el valor anterior?

Lo guardaría en una tabla aparte, donde registraría el historial de nombre y precio de productos y quien lo cambió.

Antes de guardar el nuevo nombre o precio, el backend mira que hay escrito actualmente.

En la nueva tabla se anota ese nombre o precio actual.

A continuación, se actualiza el nombre o precio del producto.

¿Guardarías esa auditoría en la misma tabla, en una tabla aparte o en otro sistema?

Como mencione anteriormente, tendría una tabla aparte llamada “auditoria” en la cual guardaría los valores editados anteriores de nombre, precio y id del producto. Dicha tabla la crearía por 3 razones:

Rendimiento: la tabla “Producto” debe ser liviana para que las búsquedas sean rápidas. Si la llenamos con versiones viejas de nombre y precio, la base se vuelve lenta.

Integridad: poder configurar dicha tabla para que solo se pueda insertar datos, impidiendo que alguien manipule datos antiguos.

Escalabilidad: si en un futuro crece demasiado el historial, podemos mover los datos de “tabla de auditoria” un sistema de almacenamiento más económico (como una base de datos de archivo).

1.3 Para pensar (no implementar)

1.3 Estrategia de Escalabilidad y Performance

Ante un escenario de crecimiento masivo (500,000 productos y miles de movimientos diarios), el sistema debe evolucionar para evitar cuellos de botella. Mi estrategia se basa en tres ejes:

1. Optimización de Consultas (Índices y Paginación)

- **Índices Estratégicos**: Implementaría índices en las columnas de mayor consulta (como producto_id, fecha y categoria_id). Esto permite que el motor de PostgreSQL localice la información de forma casi instantánea mediante estructuras de búsqueda eficientes, en lugar de realizar un escaneo secuencial de toda la tabla.
- **Paginación a nivel de API**: Para proteger la memoria tanto del servidor como del navegador, el backend nunca devolverá los 500,000 registros en una sola petición. Se implementará una paginación (por ejemplo, de 50 resultados por página) utilizando los parámetros limit y offset en las consultas SQL.

2. Particionamiento de Tablas (Eficiencia Estructural)

Para la tabla de Movimiento, que es la que más crece, aplicaría Particionamiento por Tiempo.

- **Justificación:** Si bien una consulta SQL con un filtro WHERE fecha = 'hoy' funciona, en una tabla con millones de filas el motor igual debe descartar muchísima información. Al particionar la tabla mensualmente, la base de datos "físicamente" solo lee el segmento del mes actual.
- **Transparencia:** Para el código, sigue siendo una única tabla Movimiento, pero para el disco duro, están separadas. Esto acelera las búsquedas y facilita el mantenimiento (por ejemplo, archivar datos de hace 10 años es mucho más rápido).

3. Cálculo de Stock Histórico Eficiente

Calcular el stock "en el aire" (sumando y restando movimientos desde el inicio de los tiempos) se vuelve computacionalmente inviable con el volumen planteado. Para resolverlo, propongo dos soluciones:

- **Técnica de Snapshots** (Esta es una recomendación de IA, que me pareció interesante): En lugar de procesar millones de filas, el sistema toma una "foto" del stock al cierre de cada mes y la persiste en una tabla de saldos mensuales.
 - **Ejemplo:** Si se solicita el stock al 15 de mayo, el sistema toma el snapshot del 30 de abril y solo procesa los movimientos ocurridos en los últimos 15 días. Esto reduce el procesamiento de millones de filas a solo unas pocas decenas, logrando una respuesta instantánea.
- **Vistas Materializadas:** Para reportes históricos que no cambian, utilizaría Vistas Materializadas. A diferencia de una vista común, esta guarda el resultado del cálculo físicamente en el disco.
 - **Beneficio:** Al consultar el stock de hace tres meses, la base de datos no realiza ningún cálculo; simplemente lee el dato ya procesado. Estas vistas se programarían para refrescarse automáticamente durante la noche o tras cada cierre de inventario.

2. Backend

2.1 Decisiones de arquitectura

¿Qué lenguaje y framework usarías?

Voy a usar Python como lenguaje y FastAPI como framework.

FastAPI porque crear APIs de forma muy rápida. Lo cual es muy importante para el caso que estamos planteando.

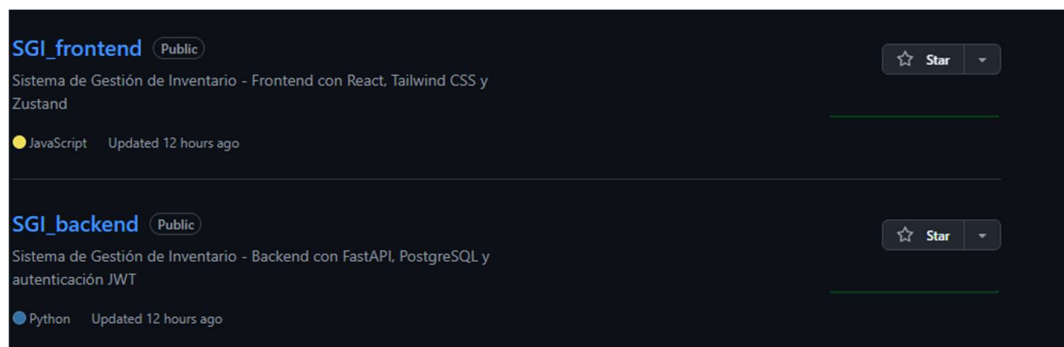
Además, FastAPI y SQModel son herramientas que fueron diseñadas para trabajar juntas sin tener que agregar o descargar nada extra.

¿Cómo organizarías el proyecto en carpetas? Prestá especial atención a cómo pensás la orquestación del proyecto: cómo se conectan las capas entre sí (rutas, servicios, repositorios, modelos), y cómo esa estructura facilita mantener y escalar el sistema.

En mi **Backend** en VScode implementaría los archivos y carpetas de esta forma:

/app	
├── /api	# Capa de Controlador (Rutas/Endpoints)
├── /services	# Capa de Servicio (Lógica de Negocio)
├── /repositories	# Capa de Datos (Consultas SQLAlchemy)
├── /models	# Definición de tablas de la DB
├── /schemas	# Modelos de validación (Pydantic)
├── /core	# Configuración global y seguridad
└── main.py	# Punto de entrada de la aplicación

En **GitHub**, haría 2 repositorios así:



Todas las carpetas mencionadas a continuación se encontrarían en el **backend**.

Rutas (API): Recibe la solicitud HTTP. No sabe nada de la base de datos; solo le pasa los datos al servicio correspondiente.

Servicios: Es el que hace la lógica y el corazón del sistema. Aquí se ejecutan las transacciones. Por ejemplo, antes de llamar al repositorio para restar stock, el servicio verifica que el stock no quede negativo.

Repositorios: Es el único que usa SQLAlchemy para hablar con la base de datos. Se encarga de buscar, guardar o actualizar registros. Osea, contiene las consultas SQL.

Modelos: Definen la estructura física de las tablas Categoría, Producto y Movimiento. Osea acá se representan las tablas de la BD en código Python.

Esta estructura facilita mantener y escalar por 2 razones:

Por un lado, porque es mucho más fácil de mantener, dado que cualquier cambio que haya que hacer, nos vamos a la respectiva carpeta y lo solucionamos, sin tener que revolver todo el proyecto.

Por otro lado, es más fácil de escalar. Al estar separado, podríamos cambiar la base de datos o agregar nuevas funciones (como alertas de stock bajo) sin tocar el código que ya funciona. O por el lado del Frontend podemos migrar a otro lenguaje sin la necesidad de tocar el Backend.

¿Dónde y cómo configurarías las reglas de negocio (ej: ¿stock mínimo, tipos de movimiento válidos)?

Las reglas de negocio se centralizarían en el **Backend**, pero distribuidas en diferentes capas:

- **Validaciones de Atributos (Capa de Modelos):**
 - Reglas estructurales como "precio no nulo" o "stock no negativo" se definen directamente en los **Modelos de SQLModel**.
 - Utilizaría restricciones para asegurar que el stock nunca sea menor a cero.
 - Esto garantiza que la regla se cumpla tanto en la validación de datos de entrada (Pydantic) como en la restricción física de la base de datos (SQL).
- **Lógica de Operación (Capa de Servicio):**
 - Reglas dinámicas o que requieren lógica compleja, como verificar si un **tipo de movimiento** es válido (Entrada/Salida) o si el usuario tiene permisos, se ubican en la **Capa de Servicio**.
 - Aquí es donde el "cocinero" del sistema decide si una operación se permite o se rechaza basándose en el estado actual del inventario.
- **Centralización de Parámetros (Capa Core/Config):**
 - Valores como el **stock mínimo** o umbrales de alerta no estarían "hardcodeados" en las funciones.
 - Usaría un archivo `app/core/config.py` o variables de entorno.
 - Esto facilita el mantenimiento: si el día de mañana la empresa cambia su política de stock mínimo, solo se modifica un valor en la configuración sin tener que tocar la lógica del programa.

2.2 Deploy

¿Cómo deployarías este sistema? ¿En la nube, Docker, empaquetado?

Mi elección principal sería utilizar **Docker** para "empaquetar" la aplicación y luego deployarla en la **nube**.

- **Docker (Contenedores):** Utilizaría contenedores para asegurar que el backend (FastAPI/SQLModel), el frontend (React) y la base de datos (PostgreSQL) corran exactamente igual en mi computadora que en el servidor. Esto evita el clásico problema de "en mi máquina funciona".
- **La Nube:** Para una pequeña empresa, elegiría servicios tipo **PaaS** (Platform as a Service) como Railway o Render, o una instancia en AWS/DigitalOcean. Estos servicios permiten escalar recursos fácilmente si el inventario llega a los 500,000 productos mencionados.
- **Empaquetado:** No usaría un empaquetado simple (como un .exe o instalador manual) porque dificulta las actualizaciones y el mantenimiento remoto que requiere un sistema de gestión web moderno.

¿Cómo manejarías las diferencias entre entorno local y producción?

Para manejar estas diferencias sin romper el sistema, aplicaría las siguientes prácticas:

- **Variables de Entorno (.env):**
 - Toda la configuración sensible (contraseñas de la base de datos, claves de la API) se guardaría en archivos .env.
 - **Local:** El backend se conecta a una base de datos PostgreSQL local dentro de un contenedor Docker para desarrollo.
 - **Producción:** El backend toma las credenciales de la base de datos productiva de la nube, sin cambiar ni una línea de código.
- **Docker Compose:**
 - Usaría archivos docker-compose.yml diferenciados.
 - En **Local**, habilitaría el "hot-reload" para que los cambios en el código se vean al instante.
 - En **Producción**, usaría imágenes optimizadas y seguras, exponiendo solo los puertos necesarios.
- **Manejo de Base de Datos:**
 - Usaría **Alembic** (migraciones) para asegurar que la estructura de las tablas (Categoría, Producto, Movimiento) se actualice en producción de forma automática y segura, manteniendo la paridad con el entorno local.

Al separar la configuración del código, garantizo que el sistema sea **portátil**. Si la pequeña empresa crece, puedo mover el contenedor de Docker a un servidor más potente en minutos.

2.3 Seguridad: roles, perfiles y contraseñas

¿Cómo manejarías los distintos roles de usuario en el sistema (por ejemplo: administrador, operador de depósito, ¿solo lectura)? ¿Dónde definirías esos permisos?

Implementaría un sistema de Control de Acceso Basado en Roles.

- **Definición de Roles:** Los definiría como un Enum en el código (dentro de la carpeta /models o /core) con los niveles solicitados como por ejemplo: ADMIN, OPERADOR y LECTURA.
- **Ubicación de los Permisos:**
 - En la Base de Datos: Cada usuario tendría un campo rol_id vinculado a su perfil.
 - En el Backend (Capa de Servicio): Definiría una matriz de permisos o decoradores que validen si el rol del usuario tiene permiso para la acción (ej: solo el ADMIN y OPERADOR pueden registrar movimientos de stock).

¿Qué consideraciones tendrías para el manejo de contraseñas? (almacenamiento, hashing, políticas mínimas.)

Usaría algoritmos de **hashing** fuertes como **bcrypt** o **Argon2** con un "salt" (un valor aleatorio único por usuario) para proteger las contraseñas.

Sobre políticas mínimas: Implementaría validaciones en el registro y cambio de clave para asegurar:

- Longitud mínima (ej: 8 caracteres).
- Uso de caracteres especiales, números y mayúsculas.
- Bloqueo temporal de cuenta tras varios intentos fallidos para prevenir ataques de fuerza bruta.

¿Cómo protegerías los endpoints según el rol del usuario autenticado?

Para proteger las rutas de la API según el usuario, usaría un esquema basado en **JWT (JSON Web Tokens)**:

- **Autenticación:** Cuando el usuario se loguea, el servidor le entrega un token JWT que contiene su ID y su Rol de forma cifrada.
- **Middleware/Dependencias en FastAPI:**
 - Crearía una **dependencia de seguridad** que intercepte cada solicitud al endpoint.
 - Esta función lee el token, verifica si es válido y extrae el rol del usuario.
 - Si un OPERADOR intenta acceder a un endpoint de "Borrar Producto" (exclusivo de ADMIN), el sistema devuelve automáticamente un error **403 Forbidden** antes de que la petición llegue a la lógica de negocio.

2.4 Logs del sistema

¿Qué tipo de eventos registrarías en los logs? (errores, acciones de usuario, cambios de estado, etc.)

Dividiría los logs en tres categorías principales para tener una visibilidad completa del sistema:

- **Errores y Excepciones (Logs de Aplicación):** Fallos de conexión a la base de datos, errores 500, o tiempos de espera agotados (timeouts).
- **Acciones de Usuario y Seguridad:** Intentos de inicio de sesión (exitosos y fallidos), creación de nuevos usuarios y cambios en los permisos de roles.

¿Dónde guardarías esos logs y con qué herramienta? ¿Archivo local, servicio externo, base de datos?

La elección depende de la criticidad y el volumen de datos:

- **Entorno Local (Desarrollo):** Usaría la salida estándar (consola) y un **archivo local** rotativo para depuración rápida.
- **Entorno de Producción (Nube):**
 - **Herramienta:** Utilizaría un sistema de gestión de logs centralizado como **ELK Stack** (Elasticsearch, Logstash, Kibana) o servicios administrados como **CloudWatch** (AWS) o **Logtail**.
 - **Por qué:** Estos servicios permiten buscar entre millones de líneas de log en segundos, crear alertas automáticas si detectan muchos errores y no consumen espacio en el disco del servidor de la aplicación.
- **Base de Datos:** No guardaría los logs de aplicación en la base de datos relacional (PostgreSQL), ya que el volumen de escritura podría afectar el rendimiento de las ventas y el inventario.

¿Cómo diferenciarías los logs de aplicación de la auditoría de negocio (que ya abordaste en la Parte 1)?

Característica	Logs de Aplicación	Auditoría de Negocio (Parte 1)
Propósito	Diagnosticar fallos técnicos y rendimiento.	Rendición de cuentas y trazabilidad de activos.
Audiencia	Desarrolladores y Administradores de Sistemas (SRE).	Dueños del negocio, auditores o contadores.
Ubicación	Archivos externos o servicios de log (CloudWatch).	Base de Datos (Tabla ProductoAuditoria).
Ejemplo	"Error 504 en el endpoint /productos".	"El usuario X cambió el precio de \$100 a \$120".
Retención	Se suelen borrar o archivar después de 30-90 días.	Deben conservarse por años por razones legales/contables.

Diferenciar estas capas es vital: mientras que los **logs de aplicación** nos ayudan a saber por qué se cayó el sistema, la **auditoría de negocio** nos permite explicarle al cliente por qué cambió el valor de su inventario

2.5 Endpoint a implementar

Endpoint que devuelva el listado de productos con su stock actual.

Cliente HTTP

↓

GET /api/productos?categoria=electronica&skip=0&limit=20

↓

[routes.py] - Endpoint HTTP (valida parámetros)

↓

[producto_service.py] - Lógica de negocio (orquestra operación)

↓

[producto_repository.py] - Acceso a datos (ejecuta SQL con JOIN)

↓

PostgreSQL (devuelve Producto + Categoría)

3. FrontEnd

3.1 Elección de tecnología

¿Cuál elegirías para este sistema y por qué?

Elegiría **React** para este sistema de gestión de inventario.

- **Escalabilidad:** React permite construir una interfaz modular basada en componentes, lo cual es ideal si el sistema crece de una gestión simple a manejar 500,000 productos.

Control Total: Ofrece una manipulación precisa del DOM y del estado de la aplicación.

Ecosistema: La disponibilidad de librerías para gráficos (como Chart.js o Recharts) facilita el requisito de visualizar el stock de forma dinámica.

¿En qué contexto usarías Streamlit? ¿Y React?

Streamlit: Lo usaría en contextos de **Ciencia de Datos o Prototipado Interno rápido**. Es ideal para herramientas donde lo más importante es mostrar datos o modelos de IA con muy poco esfuerzo de frontend, y donde la concurrencia de usuarios es baja.

React: Lo usaría en **Aplicaciones Web de grado productivo** que requieren una interacción fluida, manejo de roles, y que deben ser utilizadas por múltiples usuarios (administradores, operadores) simultáneamente.

¿Qué sacrificás con cada uno?

Con React sacrificás Simplicidad Inicial: Requiere configurar un entorno de desarrollo más complejo (Node.js, dependencias, rutas) y escribir mucho más código para tareas simples en comparación con Streamlit.

Con Streamlit sacrificás Flexibilidad y Control: Se pierde el poder de personalización detallada del diseño y el control fino sobre la lógica del navegador. Además, escalar una aplicación compleja con muchos estados interdependientes en Streamlit puede volverse difícil de mantener.

3.2 Pantallas a diseñar

¿Qué columnas mostrás? ¿Por qué esas?

Voy a mostrar datos relevantes para el usuario. Como el nombre producto, precio unitario, categoría y stock actual. El id no lo mostraría porque en el frontend que es donde el usuario ve los datos, no le es relevante el id de un producto. Si llego a querer ver el id lo haría con console.log.

Nombre del Producto: Identificación clara del artículo.

Categoría: Fundamental para que el usuario pueda agrupar visualmente productos similares (ej: "Electrónica").

Precio Unitario: Valor de referencia para ventas o valuación de inventario.

Stock Actual: El dato más crítico para la gestión de compras y ventas.

¿Cómo indicás visualmente si el stock está bajo?

En la tabla donde muestre los datos usaría:

Color de Fondo/Texto: Si el stock está por debajo del umbral mínimo, la fila o la celda del número se resalta en **rojo** (crítico) o **amarillo** (advertencia).

Etiquetas: Etiquetas con texto como "Stock Bajo" o "Sin Stock" junto al número para que sea legible incluso para personas con daltonismo.

Iconografía: Un pequeño icono de advertencia al lado del nombre del producto para captar la atención rápidamente.

¿Qué acciones tendrían desde esa vista?

Para que la tabla sea operativa y no solo de lectura, incluiría:

Filtro por Categoría: Un desplegable para limpiar la vista y enfocarse solo en una familia de productos.

Botón de "Registrar Movimiento": Un acceso directo en cada fila que abra el formulario para cargar una entrada o salida de ese producto específico.

Botón de Editar: Para corregir nombres o precios (lo que dispararía la auditoría mencionada anteriormente).

Productos

Filtrar por categoría: Todas

Nombre	Categoría	Precio	Stock	Mínimo	Acciones
televisor	electronica	\$ 500.00	5 Crítico	10	Movimiento

1 producto(s)

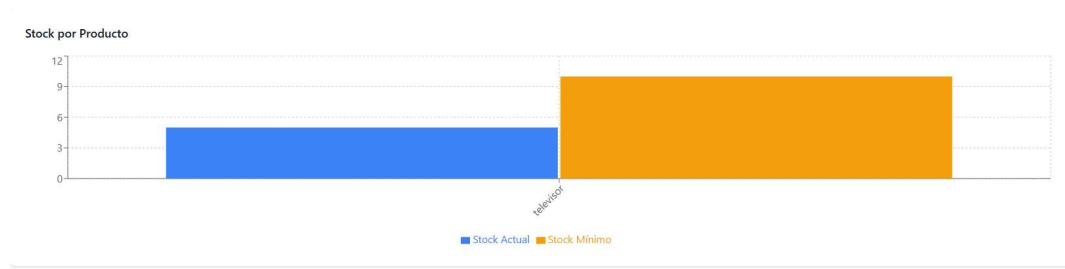
Refrescar

Incorporará un gráfico de stock por producto (ej: barras comparando el stock actual de cada producto o por categoría). ¿Qué tipo de gráfico elegirías y por qué?

Elegiría un **Gráfico de Barras Horizontales**.

Por qué: Con 500,000 productos, un gráfico circular (pie chart) sería ilegible. Las barras horizontales permiten comparar fácilmente las cantidades entre productos y leer los nombres de los artículos sin que se solapen, incluso si los nombres son largos.

Interactividad: Implementaría un filtro por categoría para que el gráfico no intente mostrar miles de barras a la vez, enfocándose en los productos más relevantes para el área que el usuario esté auditando en ese momento.



Alerta de productos con stock bajo

¿Cómo definirías ese umbral? ¿Sería fijo, configurable por categoría, configurable por producto?

El umbral sería **configurable por producto**.

Justificación: No es lo mismo que falte un "Tornillo" (donde quizás necesitas 1000 de stock mínimo) a que falte un "Transformador" (donde con 2 unidades estás bien).

¿Qué cambios necesitaría el modelo de datos para soportarlo correctamente?

Cambio en el modelo: Agregaría el campo stock_minimo a la tabla **Producto**.

¿Qué consideraciones de UX tendrías en cuenta para no generar ruido con alertas que el usuario termina ignorando?

Jerarquía de color: Usaría un color amarillo para "Stock Próximo al Límite" y rojo parpadeante o negrita solo para "Stock Crítico".

Notificaciones Inteligentes: En lugar de mostrar una ventana emergente (pop-up) por cada producto, usaría un **centro de notificaciones** o una sección de "Resumen Crítico" en el dashboard que agrupe todas las alertas, evitando interrumpir el flujo de trabajo del usuario.

Formulario para registrar un movimiento (entrada o salida):

¿Qué campos incluís?

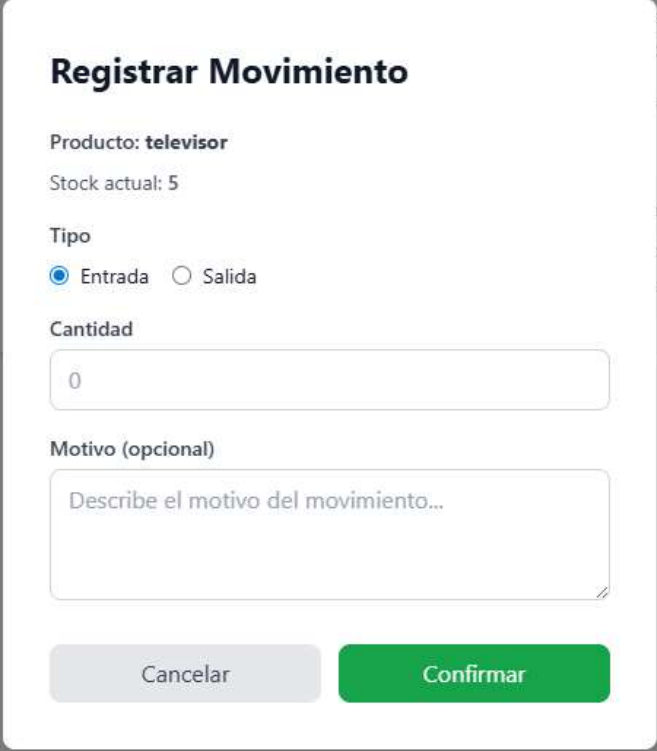
Producto (Selector/Buscador).

Tipo de Movimiento (Radio buttons: Entrada / Salida).

Cantidad (Input numérico).

Motivo (Texto breve para auditoría).

Fecha/Usuario (Se cargan automáticamente en el backend por seguridad).



The image shows a web application interface with a modal form titled "Registrar Movimiento". The form is set against a background with two tabs: "Registrar Movimiento" (active) and "Información". The form itself contains the following elements: a title "Registrar Movimiento", a label "Producto: televisor", a label "Stock actual: 5", a "Tipo" section with radio buttons for "Entrada" (selected) and "Salida", a "Cantidad" input field with the value "0", a "Motivo (opcional)" label, a text area with the placeholder "Describe el motivo del movimiento...", and two buttons at the bottom: "Cancelar" (disabled, light gray) and "Confirmar" (active, green).

¿Cómo validás que no quede stock negativo?

Frontend (UX): El botón "Guardar" se deshabilita si la cantidad de salida ingresada es mayor al stock actual disponible, mostrando un mensaje de error en rojo al instante.

Backend (Seguridad): El **Servicio** vuelve a verificar el stock en la base de datos justo antes de guardar para evitar errores de concurrencia (si dos personas intentan sacar el último producto al mismo tiempo).

¿Qué botones ponés y dónde?

Botón de "Confirmar Movimiento": A la derecha, resaltado con el color principal de la marca.

Botón de "Cancelar": A la izquierda, con un diseño más neutro para evitar clics accidentales que borren el formulario.

Ubicación: Al final del formulario, siguiendo el orden de lectura natural de arriba hacia abajo.

